Extended Abstract

Motivation Fantasy basketball is a strategic, statistics-driven game enjoyed by millions of NBA fans, with a critical emphasis placed on the initial draft phase. Human managers typically rely on heuristics and personal biases to select players, yet this process presents a well-structured, sequential decision-making problem suitable for reinforcement learning (RL). The challenge lies in exploiting the competition format, especially in categories-based leagues where performance is evaluated across multiple statistical dimensions. This project investigates whether RL agents can learn optimal drafting strategies and outperform randomized or baseline approaches.

Method The draft process is modeled as a Markov Decision Process (MDP), where each agent decision corresponds to selecting a player based on a structured observation space. Three RL algorithms-Soft Actor-Critic (SAC) adapted for discrete actions, Deep Q-Learning Networks (DQN), and Proximal Policy Optimization (PPO)—are applied to this problem. Each method incorporates action masking to enforce valid choices and uses an early action biasing strategy to guide exploration toward high-value players early in training. The environment rewards agents based on their final standing after a simulated round-robin match-up evaluation, considering wins across nine fantasy basketball statistical categories.

Implementation A custom OpenAI Gym environment simulates the draft process and opponent behavior using semi-random pick rules derived from real-world tendencies. In the environment, the RL agent competes in a 10-team snake draft, where small rewards are assigned after each draft round and larger rewards are assigned after all teams complete their rosters. Training and evaluation data are extracted from the 2024–2025 NBA season via www.basketball-reference.com.

Results Of the three methods, only SAC exceeded the baseline, achieving an average reward of 0.201—approximately equivalent to finishing in the top three of a 10-team league. DQN and PPO failed to converge effectively, showing average rewards below baseline and high variance. Further experiments revealed that reducing the number of players per team improved agent consistency and reward, with SAC achieving near-optimal results when only selecting a single player. Qualitative analysis showed that the SAC agent learned to emphasize specific categories and often surrender too many others, which led to suboptimal team compositions.

Discussion The results highlight the difficulty of modeling this adversarial, stochastic environment using conventional RL methods. The complexity introduced by longer drafts and semi-random opponent behavior likely led to policy collapse or unstable learning. The use of early action biasing and constrained exploration helped to some degree, but future improvements could include expanding the observation space or testing multi-agent training frameworks. Benchmarking against human managers was not performed but would be valuable in future assessments.

Conclusion This work demonstrates the feasibility and limitations of applying RL to fantasy basketball drafting. While SAC showed some promise in simpler scenarios, all three tested architectures struggled with the full complexity of the environment. The findings suggest that no simple, generalizable strategy guarantees draft dominance, reinforcing the balanced and competitive nature of fantasy sports. Future research can explore more advanced RL techniques, simulation of full seasons, and real-time roster adjustments to more closely mirror the challenges faced by fantasy managers.

Optimizing Fantasy Basketball Drafting with Reinforcement Learning

Arthur He
Stanford University
arthurhe@stanford.edu

Abstract

Fantasy basketball leagues are popular simulation games that allow NBA fans the opportunity to engage in strategic team-building with real-world athletes in competition against other managers. In this report, I address the challenge of optimizing player selection using reinforcement learning (RL). By modeling the draft process as a sequential decision-making problem, I aim to train an agent that can adaptively pick players to maximize the team's potential performance. My approach trains the agent based on historical player data and compares 3 different learning architectures: soft-actor critic (SAC) in a discrete action space, deep Q-networks (DQN), and proximal policy optimization (PPO). Through empirical evaluation, I demonstrate the limitations and effectiveness of each method. Additionally, I discuss the complex nature of this environment and why simple algorithms do not necessarily succeed. My results show that, although RL-based strategies can provide insight into fantasy sports, more advanced techniques may be needed to find the optimal exploitation strategies.

1 Introduction

Fantasy sports leagues are a common way for fans to enjoy professional sports with their friends by having a stake in individual player performance. Often times, managers will buy their spot in the league in hopes of winning and receiving a higher payout. Traditionally, in fantasy basketball, season-long leagues have 3 phases in common: an initial player draft to decide teams, a regular season where match-up performance decides playoff seeding, and then a tournament-style playoff period. As one might expect, drafting the best players during the team selection phase is highly important for the long-term success of a team since their real-life performance statistics determine the fantasy outcomes. This project focuses on optimizing this first draft phase with the use of an RL-trained agent.

There are different ways in which the fantasy team draft is completed, the most common being the snake draft. The snake draft is a sequential decision-making problem in which managers take turns selecting real athletes. The pick order is randomized and each manager gets one pick per round. In every subsequent round, the pick order is reversed. For example, if manager A gets the first pick in a 5-team league, manager A would then also get the tenth pick (last pick of the second round).

Player selection strategy is largely based on competition criteria as there are different types of leagues. The two most common types are points leagues and categories leagues. In points leagues, the strategy is much simpler, since all counting statistics are aggregated into a single score to be evaluated against. Categories leagues evaluate teams on the number of statistic categories a team outperforms another team in. Usually, there are nine categories (AKA 9cat): points, rebounds, assists, field goal percentage, free throw percentage, 3-pointers made, steals, blocks, and turnovers. This competition format allows strategies where managers can pick players to "punt" (i.e. surrender) certain categories to optimize for others and will be the format mimicked in our training environment.

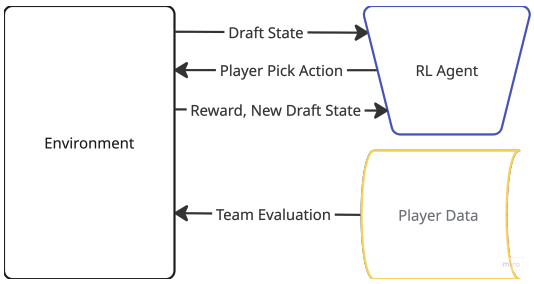

Figure 1: Reinforcement learning interactions

The objective of this project is to see if I can use common reinforcement learning techniques to train an agent that exploits this competition format. Currently, much of this game is played from a fan's perspective. In other words, human bias and heuristics are used to form teams. I am interested in how an RL agent performs given the same data. I am also interested in finding out if there is any learning architecture that excels in this environment and what patterns emerge from the winning team compositions. The training process should provide insights into policy learning in constrained, adversarial domains.

2 Related Work

Prior research in the domain of fantasy sports includes analyses of human drafting strategies [1], models built for daily performance predictions [4] and lineup construction, as well as drafting agents for various sports. However, most existing systems do not model the sequential and adversarial nature of a snake draft.

Reinforcement learning has shown promise in similar domains such as card/board games and stock market trading where long-term planning and adaptive behavior are required. Notably, AlphaZero [2] showed that a general reinforcement learning algorithm, trained through self-play, can master sequential decision-making tasks without prior domain knowledge. This highlights the potential for similar approaches to excel in fantasy draft environments where strategic foresight and opponent consideration are critical.

Furthermore, previous work has shown that the novel use of deep Q-learning (DQN) and proximal policy optimization (PPO) agents has produced winning fantasy cricket team rosters [3]. This approach, however, evaluates on a points-based scoring system in which all performance statistics are aggregated based on some arbitrary rule set (as mentioned before). This project will borrow the proposed techniques, but focus on winning opponent match-ups in a categories-based evaluation and will adjust the reward methodology accordingly.

In the space of fantasy basketball specifically, there exists ML research completed for drafting/team building [8] [9], but none that implement reinforcement learning methods under the same environment conditions.

3 Method

3.1 Reinforcement Learning Framework Definitions

For this project, the fantasy draft sequential decision-making problem is constructed as a Markov Decision Process (MDP). Data is generated in the form of a tuple (s, a, r, s') where s denotes the current state of the environment, a denotes the action taken by the agent, r denotes the reward received after taking action a in state s, and s' denotes the new state of the environment after the action is taken.

State Space: At each timestep t, the agent observes the current state $o_t \in s$, which provides the necessary context for making a decision at that specific moment. The observation encapsulates current available players, the current aggregate category statistics for each team, the agent's current league ranking and pick position. Specifically, the state o_t is expressed as $o_t = (P_t, D_t, W_t, Y)$,

where P_t represents the current player availability vector, D_t represents the current team category statistics, W_t represents the current league ranking, and Y represents the RL agent's pick position.

Action Space: At each timestep t, the agent performs action a_t given o_t in the form of a scalar value representing the index of a player in P_t. P_t is a vector of binaries with 1 signifying that the player is available to be drafted and 0 signifying that the player has already been drafted (i.e. unavailable). By selecting a player index, the RL agent creates a new state o_{t+1} with an immediate round reward r_t. P_t, D_t, and W_t are all updated as well. The RL agent's goal is to discover an optimal policy that maximizes the total cumulative reward across rounds in the episode. The action selection process is guided by one of the three policy learning algorithms defined in section 3.2.

In all three algorithms, an action availability mask was applied to disallow selecting unavailable players, thus removing any negative rewards. The masked action probability $\tilde{p}_t$ distribution given p_t and the player availability vector P_t is defined as:

$$\tilde{p}_t = \frac{p_t \odot P_t}{\sum_{i=1}^{n} p_t[i] \cdot P_t[i]} \tag{1}$$

Another technique used across algorithms was early action biasing. The player data used is already sorted by Yahoo's average draft position (ADP) which sorts better overall players towards the front indices of the action vector. To help the agent gain a positive signal earlier in training with less useless exploration, a decaying bias was applied to the action probabilities as defined:

$$\tilde{p}_t^* = \frac{\tilde{p}_t \odot d}{\sum_{i=1}^{n} \tilde{p}_t[i] \cdot d[i]} \tag{2}$$

where

$$d[i] = \exp\left(-\frac{i}{\tau}\right), \quad \text{for } i = 0, 1, \ldots, n-1 \tag{3}$$

The decay constant τ can be any value $\tau > 0$ (tuned to $\tau = 30$ in this project).

Environment and Reward Definitions: Detailed in section 4.

3.2 Agent Architectures

Three popular reinforcement learning algorithms were chosen and deployed because of their track record of success in similar environments:

1. Soft actor-critic (SAC) in a discrete action space
2. Deep Q-networks (DQN)
3. Proximal policy optimization (PPO)

Since this project's code was implemented by following the well-known papers introducing each algorithm, below are their brief descriptions.

Soft actor-critic (SAC): SAC is an off-policy actor-critic algorithm encourages both performance and exploration [11]. While originally formulated for continuous action spaces, SAC has been adapted to discrete settings by modeling the policy as a categorical distribution over actions.

SAC optimizes a stochastic policy $\pi(a \mid s)$ by maximizing the entropy-augmented objective:

$$J(\pi) = \mathbb{E}_{(s_t, a_t) \sim \mathcal{D}} \left[Q(s_t, a_t) - \alpha \log \pi(a_t \mid s_t) \right] \tag{4}$$

$\alpha > 0$ is the temperature parameter that controls the trade-off between reward maximization and entropy. This was tuned in experimentation to $\alpha = 0.3$.

In discrete SAC, the critic maintains two Q-functions Q_1 and Q_2. The target value function is computed using the policy's current probabilities and both Q-networks:

$$V(s_{t+1}) = \sum_{a'} \pi(a' \mid s_{t+1}) \left[\min\left(Q_1(s_{t+1}, a'), Q_2(s_{t+1}, a')\right) - \alpha \log \pi(a' \mid s_{t+1}) \right] \tag{5}$$

The target Q-value is then:

$$y_t = r_t + \gamma(1 - d_t)V(s_{t+1}) \tag{6}$$

The critic networks are updated by minimizing the Bellman error:

$$\mathcal{L}_{\text{critic}} = \mathbb{E}_{(s_t,a_t,r_t,s_{t+1})\sim\mathcal{D}}\left[(Q_i(s_t,a_t) - y_t)^2\right], \quad i \in \{1,2\} \tag{7}$$

The actor is updated using the soft policy improvement step, which minimizes:

$$\mathcal{L}_{\text{actor}} = \mathbb{E}_{s_t\sim\mathcal{D}}\left[\sum_a \pi(a \mid s_t)\left(\alpha \log \pi(a \mid s_t) - \min(Q_1(s_t,a), Q_2(s_t,a))\right)\right] \tag{8}$$

This encourages the policy to prefer actions with high Q-values, while maintaining high entropy for exploration.

Deep Q-learning networks (DQN): DQN is an off-policy algorithm that approximates the optimal action-value function $Q^*(s,a)$ using a neural network. [12]. The learning process involves minimizing the difference between a prediction Q-network $Q(s,a;\theta)$ and a separate target network $Q(s,a;\theta^-)$ with periodically updated parameters θ^-. The temporal-difference (TD) target for training is:

$$y = r + \gamma \max_{a'} Q(s',a';\theta^-) \tag{9}$$

The loss function minimized during training is the mean squared error (MSE) between the predicted and target Q-values:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(Q(s,a;\theta) - y\right)^2\right] \tag{10}$$

where $\mathcal{D}$ is the experience replay buffer.

The weights θ are updated via gradient descent on $\mathcal{L}(\theta)$, and the target network is periodically synchronized with the policy network $\theta^- \leftarrow \theta$.

Proximal policy optimization (PPO): PPO [10] is a family of policy gradient methods designed to improve stability and performance by avoiding overly large policy updates. It does this by modifying the standard policy gradient loss with a clipped surrogate objective that penalizes updates which move the new policy too far from the old one. Explained, let:

- π_θ be the current policy parameterized by θ,
- $\pi_{\theta_{\text{old}}}$ be the policy used to collect data (fixed during update),
- A_t be the estimated advantage at time step t,
- $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ be the probability ratio.

PPO optimizes the following surrogate objective to avoid large policy updates:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\left[\min\left(r_t(\theta)A_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t\right)\right] \tag{11}$$

This clipping operation ensures the new policy does not diverge significantly from the old policy. PPO uses a value function to estimate returns, trained by minimizing the squared error:

$$L^{\text{VF}}(\theta) = \frac{1}{2}\left(V_\theta(s_t) - R_t\right)^2 \tag{12}$$

where R_t is the target return, which can be computed using Generalized Advantage Estimation (GAE). To encourage exploration, an entropy term is added:

$$L^{\text{ENT}}(\theta) = \mathbb{E}_t\left[\mathcal{H}[\pi_\theta(\cdot \mid s_t)]\right] \tag{13}$$

The final PPO objective combines all terms:

$$L^{\text{PPO}}(\theta) = -L^{\text{CLIP}}(\theta) + c_1 L^{\text{VF}}(\theta) - c_2 L^{\text{ENT}}(\theta) \tag{14}$$

where c_1 and c_2 are coefficients for the value and entropy terms.

4 Experimental Setup

4.1 Environment and Transitions

The drafting and evaluation logic explained in the introduction was incorporated into learning through a Gym environment (open source Python library developed by OpenAI for comparing reinforcement learning algorithms) for all training experiments. The environment keeps track of team rosters and defines one round (i.e. step) of the draft as processing the agent's action/pick, then automatically processing the rest of the teams' semi-random picks in snake order.

In a ADP-sorted list of players, the semi-random pick behavior of non-agent teams are as defined:
1. If the team has less than 2 players, select a random player out of the next 5 available players.
2. If the team has less than 5 players, select a random player out of the next 10 available players.
3. If the team has at least 5 players, select a random player out of the next 30 available players.

This pick behavior somewhat captures how a real draft would look and avoid the transition function being deterministic. During experimentation, these top-n-random numbers were relaxed to see if environment parameters were inducing performance noise. This is explained later in the results and discussion sections.

In the training loop, draft rounds are iterated on until the predefined team size criteria is met. The observation, action, reward, and next observation are generated and recorded in each round. Periodically, evaluation cycles are performed to record average reward (performance of the agent).

4.2 Training and Evaluation Data

Player statistics used for training and evaluation were retrieved from www.basketball-reference.com. Many similar projects have either scraped this data from a fantasy platform (e.g. ESPN, Yahoo, DraftKings) [6] [8] or used www.basketball-reference.com [5]. The data found across these websites is generally the same and reflects official NBA data. The experiments described in this report uses data collected from the 2024-2025 season.

4.3 Reward Design

The evaluation and reward structure, in detail, are defined as follows:
1. Aggregate player statistics in each category for each team
2. Compare team statistics against each other in a round-robin to generate a win-loss-tie record
3. Compute win percentage from win-loss-tie record
4. Reward team if win percentage is within the top 3 of all teams

Per-round rewards were introduced early on in an attempt to increase reward density and provide better positive signal. Negative rewards for selecting unavailable players were removed as explained in the methods. Reward values were tuned so that episode rewards were bounded between $[0, 1)$ for a 13 round draft. After tuning, completed draft rewards were 0.6 for first place, 0.4 for second place, and 0.2 for third place.

The baseline performance chosen for comparison is the average reward achieved for any given team if all teams picked randomly. This is expressed as:

$$\mathbb{E}[R] = P_1 R_1 + P_2 R_2 + P_3 R_3 \tag{15}$$

where P_1 is the probability of achieving first place and R_1 is the reward for achieving first place. In a 10-team league with the previously defined rewards, this equals 0.12.

5 Results

5.1 Quantitative Evaluation

Table 1 summarizes the average reward and standard deviation for the three different methods. Both DQN and PPO yielded below-baseline performance, which either indicates some sort of policy collapse or an impossibly difficult environment. SAC performed the best, exceeding baseline, with the highest average reward of 0.201 - meaning that it did manage to average around 3rd place results.

Table 1: 10 Team, 13 Person Roster Performance

Method	Avg Reward	Std Dev
Baseline	0.120	0.0
SAC	0.201	0.247
DQN	0.013	0.021
PPO	0.054	0.092

Table 2: SAC experimentation with different roster sizes

Roster Size	Avg Reward	Std Dev
Baseline	0.120	0.0
Teams of 1	0.573	0.052
Teams of 2	0.447	0.064
Teams of 3	0.384	0.211

Surprisingly, however, the standard deviations were all greater than the averages which suggests that there were large fluctuations in performance, episode to episode. As a reminder, rewards are left bounded by 0, so the large standard deviation means that the distribution is skewed with a long right tail. In this initial experimentation, it became clear that the problem space was more complex than anticipated and that I needed to give the agent a simpler version of this draft simulation.

Further experimentation with the roster size of each team was completed with SAC (Table 2). In these simulations, average rewards for teams of 1 converged to 0.573 which is very close to the maximum (1st place) reward of 0.6. The standard deviation on this run was also relatively quite small, indicating a more consistent policy. With each additional roster spot, however, rewards quickly declined towards the results of the previous experiment. This demonstrated that the SAC architecture was functioning as expected, but the random nature of non-agent team picks was amplified with larger rosters and made the problem much harder to solve.

5.2 Qualitative Analysis

Figures 2 and 3 illustrate the aforementioned numerical results. It is evident that the simplest (teams of 1) trial yielded converging, desirable results, whilst the rest of the trials either didn't converge or had high noise. Expanding the training network breadth and depth (hidden network size, number of layers) produced results with somewhat lower variance, but echoed this sentiment (smaller roster sizes leading to higher average reward).

In an experiment with teams of 7, an example of a typical evaluation lineup looks like:

1. Joel Embiid
2. Chet Holmgren
3. Dejounte Murray
4. Scottie Barnes
5. Jalen Williams
6. Jaren Jackson Jr.
7. Rudy Gobert

Based on the player statistics, it seems these players were highly valued by the agent for their category performances in field goal percentage, rebounds, and blocks. These players, however, are also characterized by the low number of games they played this season (which affects many of the other categories). Even though player statistics were normalized by the number of games played, it appears the agent still fell into this mediocre-reward local minimum.

Another example of this strategy-silo effect happening was with a common lineup when the first agent pick was Shai Gilgeous-Alexander. The roster would consist of players that excelled in the field goal percentage and free throw percentage category, at the expense of durability (games played)

Figure 2: Average reward plots for the three architectures.

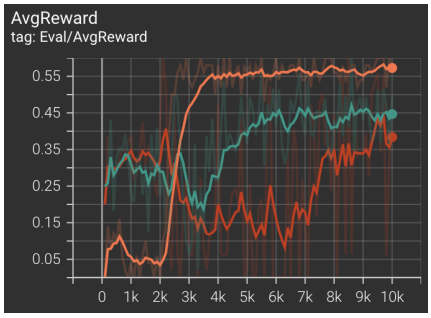

Figure 3: Roster size experimentation (orange=teams of 1, green=teams of 2, red=teams of 3).

as well as blocks, rebounds, etc. These learned strategies to focus on a few categories while hard punting the rest eventually evaluated poorly which is an interesting insight in and of itself.

The resulting analysis drawn from these experiments is that the three agent training methods presented are unable to accurately explore and model this entire problem space due to the increasing randomness introduced by additional team size.

6 Discussion

This project only tested three common RL agent training methods in the fantasy basketball draft environment and does not give insight into the many other methods that exist. More advanced methods such as adversarial (multi-agent) training were omitted due to experimental roadblocks and time constraints. Poor algorithmic performance led to much time spent debugging code and introducing many smaller algorithmic adjustments (e.g. early action biasing). In retrospect, perhaps further extending the observation space could have induced a larger performance boost, although this is yet to be tested.

Additionally, agent performance was only benchmarked against theoretical performance, as opposed to human performance. To reach a deeper understanding and better gauge of relative performance, these agents should be matched against human opponents.

Another limitation of this project was the player data collected. The data used for training and evaluation were the same. If better results were to have been observed, a more predictive approach would have been taken (evaluating based on the following year's player performance data). This would better mimic what managers do each fantasy season.

7 Conclusion

In this report, I presented three RL agent training approaches to fantasy basketball team selection using deep reinforcement learning. The discrete SAC architecture showed more promise than DQN and PPO, with the caveat that all three architectures suffered significant performance decline with

increasing randomness. Although agent performance did not impress, I offer insight and analysis into these shortcomings that may be used for further iteration.

One key implication of this work is that fantasy managers can be assured there is no easy solution or exploit for this environment. Despite the possibility of a higher performing agent, there still exists winning chances for all managers, which should inspire continued participation in this activity.

Future work could explore many of the other RL architectures and techniques to optimize agent performance in this environment. For instance, an all-agent league simulation could highlight novel strategies and player-strategy associations. Additionally, simulating the regular or playoff fantasy season with daily lineups and player adds/drops/trades would be a welcomed extension to this research.

References

[1] Smith, B. et al. (2006). Decision making in online fantasy sports communities. Interactive Technology and Smart Education, vol. 3, no. 4, pp. 347–360.

[2] Silver, D. et al. (2017). Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm. arXiv preprint, arXiv:1712.01815.

[3] Bhattacharjee, S. et al. (2024). Optimizing Fantasy Sports Team Selection with Deep Reinforcement Learning. arXiv preprint, arXiv:2412.19215v1.

[4] Ocak, I. (2020). Fantasy Basketball Prediction Using Machine Learning, GitHub repository, https://github.com/iocak28/Fantasy_Basketball_ML

[5] Mohebban, S. (2020). Machine Learning in Fantasy Basketball: Data Collection Using BeautifulSoup. Medium. Retrieved from https://medium.com/@HeeebsInc/using-machine-learning-to-predict-daily-fantasy-basketball-scores-part-i-811de3c54a98.

[6] Bhat, S. (2021). Fantasy or Not: Reinforcement Learning and Fantasy Football? Medium. Retrieved from https://medium.com/codex/fantasy-or-not-reinforcement-learning-and-fantasy-football-part-1-9e4de960c891.

[7] Hu, J., Wellman, M.P. (2003). Nash Q-Learning for General-Sum Stochastic Games. Journal of Machine Learning Research, vol. 4, pp. 1039-1069.

[8] Veluru, V. et al. (2024). Machine Learning Optimization Model to Predict Fantasy Basketball Teams. 2024 International Conference on Computing and Data Science (ICCDS), Chennai, India, 2024, pp. 1-4.

[9] Chan, Hu, Shivakumar (2015). Learning to Turn Fantasy Basketball Into Real Money. Retrieved from https://shreyasskandan.github.io/files/report-ChanHuShivakumar.pdf.

[10] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal Policy Optimization Algorithms. arXiv preprint, arXiv:1707.06347.

[11] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. (2018). Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. arXiv preprint, arXiv:1801.01290.

[12] Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. Nature, 518(7540), 529–533.

A Additional Experiments

Not mentioned in the main body of this report were hyperparameter-tuning experimentation runs as well as the introduction of a win-buffer. The win-buffer (comprised of positively rewarded steps), separate from the normal replay buffer, was added into the SAC implementation to ensure a sampling of good behavior during learning. This resulted in marginally better performance for the SAC runs.